

Computacíon Paralela y Distribuída

2026-I

José Fiestas

4 de junio de 2026

Universidad de Ingeniería y Tecnología
jfiestas@utec.edu.pe

Práctica Dirigida 04:

Unidad 4: OMP

PD04: 5 pts (avance presencial)

Tarea 4: +1 pt (entrega final)

Sistema de evaluación 2026-I

Evaluación Contínua (EC_1, EC_2):

Se desarrolla en **grupos de 2 o 3**, en clase de Laboratorio. Cada evaluación consiste de:

- una **Practica Dirigida (PD)**, que se entrega el mismo día de la evaluación, al final de la clase. Debe desarrollar las preguntas planteadas y completar el 80 % de la nota
- una **Tarea**, que se entrega unos días después de la PD, y debe completar el desarrollo de los ejercicios planteados, por el 20 % restante de la nota

En el **Anexo Detallado de Evaluaciones** en Canvas se indica el puntaje para cada forma de evaluación y los temas
Utilice las **herramientas computacionales vistas en clase**. Si utiliza fuentes externas debe citarlas, además de describir la funcionalidad de las herramientas consultadas.

Ejercicio 1: KNN (2 pts)

Paralelice el código KNN adjunto en Canvas *(knn.cpp)* usando directivas OMP.

1.1) Decida la estrategia de paralelismo (¿Cuál bucle paraleliza?) y fundamente la elección. Genere el código **(0.5 pts)**
Respuesta: Se decide paralelizar el bucle de la data de prueba N_{test}. Ya que cada data de prueba debe iterar sobre N_{train} completo, se intenta evitar un paso posterior de reducción de distancias. Código: *knn_omp/cpp*

1.2) Mida tiempos de ejecución y genere una gráfica T_{ejec} vs p para p=$\{$ 2,4,8,16$\}$. Si usa Khipu, puede llegar a p=32. Analice gráficamente escalabilidad fuerte con estos resultados **(0.5 pts)**

1.3) Decida si incrementar N_{test} o N_{train} (para medir escalabilidad débil) y mida tiempos de ejecución. Genere las gráficas necesarias y realice un análisis de escalabilidad débil **(1 pts)**

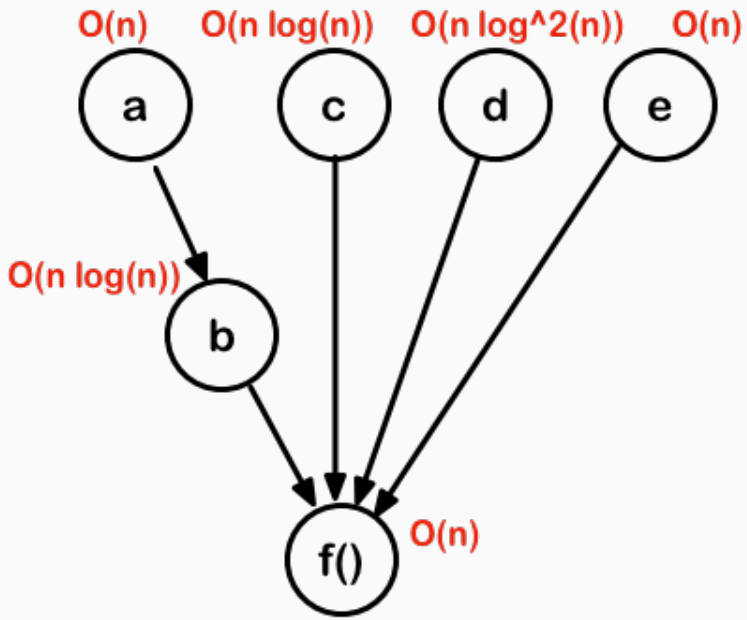
Ejercicio 2: Tareas en paralelo (2 pts)

Dada la siguiente función secuencial

```
int ejercicio_2(int n, double* v, double& x, double& y){
    int i,j,k=0;
    double a,b,c,d;
    a = calcula_a(v,x);   // O(n)
    b = calcula_b(v,a);   // O(n log(n))
    c = calcula_c(v,x);   // O(n log(n))
    d = calcula_d(v,y);   // O(n log^2(n))
    e = calcula_e(v,x);   // O(n)
    x += a + b + c;
    y += c + d + e;
    for (i=0; i<n; i++) {
          j = f(x, y, v[i]);   // O(n)
             if (j>0 && j<4) k++; }
    return k; }
```

Ejercicio 2: Tareas en paralelo (cont.)

2.1) Dibuje el grafo de dependencias (DAG) **(0.5 pts)**
Las funciones internas no modifican sus argumentos.
Se paralelizan las 5 primeras tareas y el bucle for



Se entiende que el cálculo de x, y se hace en la última tarea (no modifica el O(n))

PRAM Ejercicio 2

2.2) Diseñe un algoritmo (pseudocódigo PRAM) que maximice el nivel de paralelismo **(1 pt)**

Algorithm 1: Ejercicio 02

```
1  Input: entero n, arrays v, x, y
2  Output: contador k para valores de 0 < j < 4
3  calcula_a || calcula_c || calcula_d || calcula_e ;      // O(n log^2 n)
4  calcula_b ;                                             // O(n)
5  x += a + b + c
6  y += c + d + e
7  for i = 1 to n pardo
8  |   j = f(x, y, v[i]) ;                                 // O(n)
9  |   if (j > 0 && j < 4)
10 |   k++
```

2.3) Indique la complejidad del algoritmo obtenido **(0.5 pts)**

De acuerdo al PRAM, $T_\infty = O(n \log^2 n + n)$

Aplicando Brent, el tiempo paralelo total es

$$T_p = O(\frac{n^2}{p} + n \log^2 n + n)$$

Ejercicio 3: Función Promedio (2 pts)

La función determina las posiciones de los valores mayores al promedio en una matriz

```
int ejercicio_1(int N, double** A, double posiciones[][2]){
    int i,j,k=0;
    /* Calculo del promedio */
    for (i=0;i<N;i++) {
        for (j=0;j<N;j++)
            sum+=*(*(A+i)+j);
     }
    prom=sum/(N*N);
    /* Una vez localizado el promedio, guardar las posiciones
    de valores mayores al promedio */
    for (i=0;i<N;i++) {
        for (j=0;j<N;j++) {
            if (A[i][j] > prom){
                posiciones[k][0]=i;
                posiciones[k][1]=j;
                k = k+1;
                }
        }
    }
```

Ejercicio 3: Función Promedio (cont.)

3.1) Paralelice la función con OMP **(0.5 pts)**

3.2) Describa y analice el nivel de paralelismo logrado **(0.5 pts)**

3.3) Formule ecuaciones para el overhead en este algoritmo y obtenga una expresión final para $T_p(n, p)$ **(1 pts)**

Ejercicio 3 OMP (sol.)

```
int ejercicio_3_omp(int N, double** A, double posiciones[][2]){
    int i, j, k = 0;
    double sum = 0.0, prom;
    /* Calculo paralelo de la suma */
    #pragma omp parallel for collapse(2) reduction(+:sum)
    for (i = 0; i < N; i++)
        for (j = 0; j < N; j++)
            sum += A[i][j];
    prom = sum / (N * N);
    /* Guardar posiciones mayores al promedio */
    #pragma omp parallel for collapse(2)
    for (i = 0; i < N; i++) {
        for (j = 0; j < N; j++) {
            if (A[i][j] > prom) {
                int idx;
                #pragma omp atomic capture
                idx = k++;
                posiciones[idx][0] = i;
                posiciones[idx][1] = j;
            } } }
    return k;
}
```

Ejercicio 3 OMP (sol.)

3.2) Describa y analice el nivel de paralelismo logrado **(0.5 pts)**
En el cálculo del promedio se usa `reduction(+:sum)` , porque varios hilos acumulan parcialmente la suma. En la segunda fase, la variable `k` es compartida `atomic capture` para que cada hilo obtenga una posición única en el arreglo `posiciones`.

Con p hilos, se logra:

$$T_p = O\left(\frac{N^2}{p}\right) + overhead$$

El uso de `atomic capture` puede generar contención si muchos elementos son mayores al promedio. Sin embargo, evita condiciones de carrera y mantiene correcta la escritura en `posiciones`.

Ejercicio 3 OMP (sol.)

3.3) Formule ecuaciones para el overhead en este algoritmo y obtenga una expresión final para $T_p(n, p)$ **(1 pts)**
El algoritmo paralelo introduce overhead debido a la creación, sincronización y coordinación de los hilos. El tiempo paralelo puede expresarse como:

$$T_p = O\left(\frac{N^2}{p}\right) + T_{overhead}$$

Cada región paralela:

```
#pragma omp parallel for
```

requiere la creación o activación de hilos, distribución de iteraciones, sincronización implícita al finalizar el bucle. Este costo puede modelarse como (latencia constante de sincronización):

$$O(p)$$

Ejercicio 3 OMP (sol.)

```
reduction(+:sum)
```

cada hilo mantiene una copia privada de `sum` y al finalizar se combinan los resultados parciales. I.e.

$$O(\log p)$$

La instrucción:

```
#pragma omp atomic capture
idx = k++;
```

introduce serialización parcial porque múltiples hilos compiten por acceder a la variable compartida k. Si M elementos cumplen:

$$A[i][j] > prom$$

el costo total del atomic puede aproximarse por $O(M)$, aunque con una constante elevada debido a contención. En el peor caso $M = N^2$ por lo que el overhead atómico puede crecer hasta $O(N^2)$ aunque normalmente menor que el trabajo computacional total.

Ejercicio 3 OMP (sol.)

Considerando overhead:

$$T_p = O\left(\frac{N^2}{p} + \log p + M\right)$$

donde:

$$M = \text{cantidad de elementos mayores al promedio}$$